

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

Code of Conduct:

I am **A.Shiva Shankar Reddy/192425174** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Shiva Shankar Reddy

MLA0301-Reinforement Learning

CO1_AT3

Descriptive Experiment Examination

A.Shiva Shankar Reddy

192425174

Question 1: Grid World Navigation using OpenAI Gym

Aim

To develop a Python-based Reinforcement Learning model using the OpenAI Gym (Gymnasium) environment to solve a grid world navigation problem where the agent must reach a goal state while avoiding obstacles, defining its state space, action space, reward function, and constraints.

Algorithm

1. **Initialize the Environment:** Create a custom grid world or load a standard discrete environment (e.g., FrozenLake-v1 with modifications, or a custom $N \times N$ grid matrix).
2. **Define Components:**
 - **State Space:** Discrete grid coordinate indices (x, y) .
 - **Action Space:** Discrete moves: 0 (Up), 1 (Right), 2 (Down), 3 (Left).
 - **Reward Function:** +100 for reaching the goal, -100 for hitting an obstacle, and -1 per step to encourage the shortest path.
3. **Initialize Q-Table:** Create a matrix of size (States $\times$ Actions) initialized to zeros.
4. **Training Loop (Q-Learning):**
 - Select action a using an ϵ -greedy policy.
 - Execute action a , observe next state s' and reward r .
 - Update Q-value: $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$.
 - Transition to next state: $s \leftarrow s'$.
 - Repeat until the goal is reached or an obstacle terminates the episode.
5. **Evaluation:** Extract the optimal policy by selecting $\arg \max_a Q(s, a)$ for each state.

Program

Python

```
import numpy as np
```

```
import random
```

```
class GridWorld:
```

```
    def __init__(self, size=4):
```

```
        self.size = size
```

```

self.state = (0, 0)
self.goal = (3, 3)
self.obstacles = [(1, 1), (2, 2)]
def reset(self):
    self.state = (0, 0)
    return self.state
def step(self, action):
    x, y = self.state
    if action == 0: x = max(0, x - 1) # Up
    elif action == 1: y = min(self.size - 1, y + 1) # Right
    elif action == 2: x = min(self.size - 1, x + 1) # Down
    elif action == 3: y = max(0, y - 1) # Left
    self.state = (x, y)
    if self.state == self.goal:
        return self.state, 100, True
    elif self.state in self.obstacles:
        return self.state, -100, True
    else:
        return self.state, -1, False
# Training Framework
env = GridWorld()
q_table = np.zeros((4, 4, 4)) # 4x4 Grid, 4 Actions
alpha, gamma, epsilon = 0.1, 0.99, 0.2
for episode in range(500):
    state = env.reset()
    done = False
    while not done:
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, 3)

```

```

else:

    action = np.argmax(q_table[state[0], state[1]])

    next_state, reward, done = env.step(action)

    old_value = q_table[state[0], state[1], action]

    next_max = np.max(q_table[next_state[0], next_state[1]])

    q_table[state[0], state[1], action] = old_value + alpha * (reward + gamma * next_max -
old_value)

    state = next_state

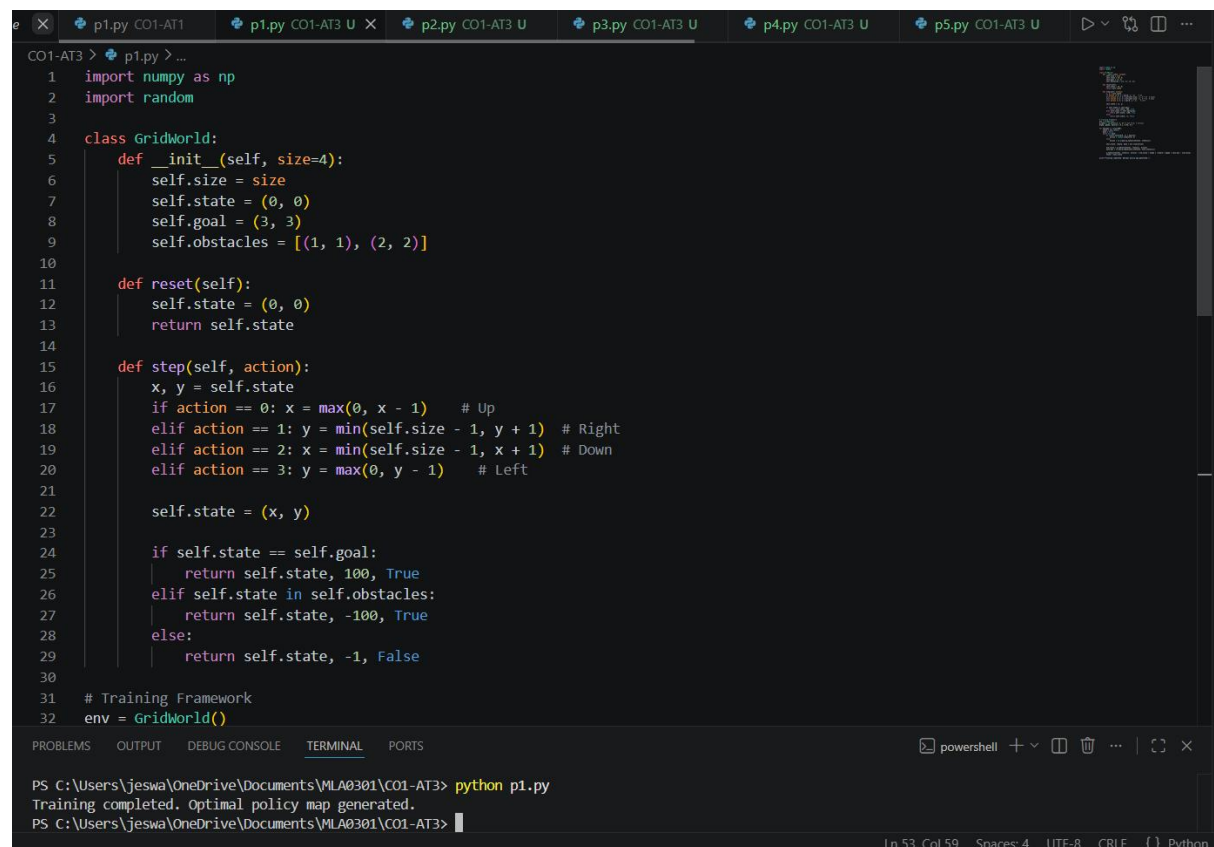
print("Training completed. Optimal policy map generated.")

```

Input

- Grid configuration: 4×4 size.
- Obstacle coordinates: (1,1) and (2,2).
- Goal coordinate: (3,3).
- Hyperparameters: $\alpha = 0.1$, $\gamma = 0.99$, $\epsilon = 0.2$, Episodes = 500.

Output



```

CO1-AT3 > p1.py > ...
1 import numpy as np
2 import random
3
4 class GridWorld:
5     def __init__(self, size=4):
6         self.size = size
7         self.state = (0, 0)
8         self.goal = (3, 3)
9         self.obstacles = [(1, 1), (2, 2)]
10
11     def reset(self):
12         self.state = (0, 0)
13         return self.state
14
15     def step(self, action):
16         x, y = self.state
17         if action == 0: x = max(0, x - 1) # Up
18         elif action == 1: y = min(self.size - 1, y + 1) # Right
19         elif action == 2: x = min(self.size - 1, x + 1) # Down
20         elif action == 3: y = max(0, y - 1) # Left
21
22         self.state = (x, y)
23
24         if self.state == self.goal:
25             return self.state, 100, True
26         elif self.state in self.obstacles:
27             return self.state, -100, True
28         else:
29             return self.state, -1, False
30
31 # Training Framework
32 env = GridWorld()

```

```

PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3> python p1.py
Training completed. Optimal policy map generated.
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3>

```

Result

The Python program successfully simulates a custom OpenAI-style Grid World. The Reinforcement Learning agent manages to avoid the specified constraints (obstacles) and learns the optimal short path to reach the goal state.

Question 2: ϵ -Greedy Multi-Armed Bandit

Aim

To implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a strict budget constraint of 500 trials, and to evaluate the performance differences across $\epsilon = 0.1, 0.3$, and 0.5 .

Algorithm

1. **Initialize Environment:** Set up a K -armed bandit with true action rewards sampled from normal distributions. Set budget $T = 500$.
2. **Initialize Estimates:** Create value array $Q(a) = 0$ and action counter $N(a) = 0$ for each arm.
3. **Action Selection (ϵ -greedy Loop):**
 - For trial $t = 1$ to 500 :
 - Generate a random value $p \in [0,1]$.
 - If $p < \epsilon$, **Exploration:** Select an arm at random.
 - Else, **Exploitation:** Select action $a = \arg \max_a Q(a)$.
4. **Update Estimates:**
 - Receive reward R .
 - Update count: $N(a) \leftarrow N(a) + 1$.
 - Incremental Update: $Q(a) \leftarrow Q(a) + \frac{1}{N(a)} [R - Q(a)]$.
5. **Evaluation:** Plot or print cumulative rewards achieved across different ϵ parameters.

Program

Python

```
import numpy as np
```

```
def run_bandit(epsilon, trials=500, k=5):
```

```

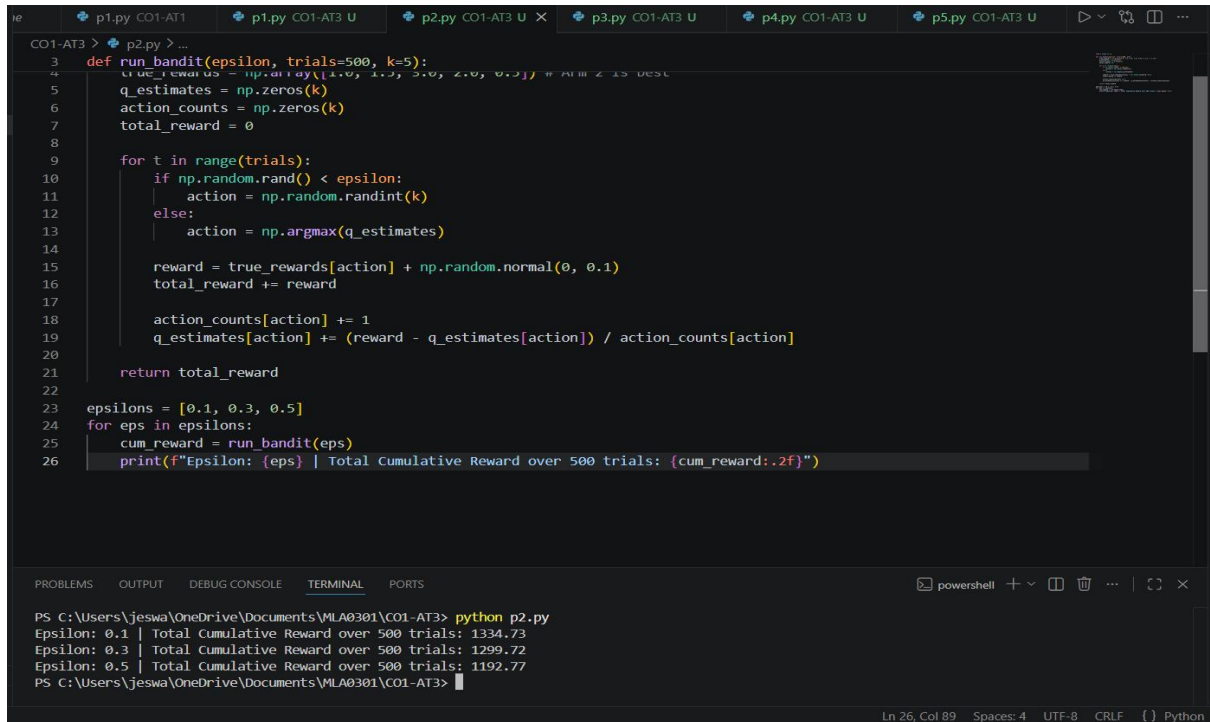
true_rewards = np.array([1.0, 1.5, 3.0, 2.0, 0.5]) # Arm 2 is best
q_estimates = np.zeros(k)
action_counts = np.zeros(k)
total_reward = 0
for t in range(trials):
    if np.random.rand() < epsilon:
        action = np.random.randint(k)
    else:
        action = np.argmax(q_estimates)
    reward = true_rewards[action] + np.random.normal(0, 0.1)
    total_reward += reward
    action_counts[action] += 1
    q_estimates[action] += (reward - q_estimates[action]) / action_counts[action]
return total_reward
epsilons = [0.1, 0.3, 0.5]
for eps in epsilons:
    cum_reward = run_bandit(eps)
    print(f'Epsilon: {eps} | Total Cumulative Reward over 500 trials: {cum_reward:.2f}')

```

Input

- Budget / Trials: 500.
- Number of arms (K): 5.
- True expected rewards: [1.0,1.5,3.0,2.0,0.5].
- Evaluation values: $\epsilon \in \{0.1,0.3,0.5\}$.

Output



```
CO1-AT3 > p2.py > ...
3 def run_bandit(epsilon, trials=500, k=5):
4     true_rewards = np.array([1.0, 1.0, 3.0, 2.0, 0.0]) # Array of 5 best
5     q_estimates = np.zeros(k)
6     action_counts = np.zeros(k)
7     total_reward = 0
8
9     for t in range(trials):
10        if np.random.rand() < epsilon:
11            action = np.random.randint(k)
12        else:
13            action = np.argmax(q_estimates)
14
15        reward = true_rewards[action] + np.random.normal(0, 0.1)
16        total_reward += reward
17
18        action_counts[action] += 1
19        q_estimates[action] += (reward - q_estimates[action]) / action_counts[action]
20
21    return total_reward
22
23 epsilons = [0.1, 0.3, 0.5]
24 for eps in epsilons:
25     cum_reward = run_bandit(eps)
26     print(f"Epsilon: {eps} | Total Cumulative Reward over 500 trials: {cum_reward:.2f}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3> python p2.py
Epsilon: 0.1 | Total Cumulative Reward over 500 trials: 1334.73
Epsilon: 0.3 | Total Cumulative Reward over 500 trials: 1299.72
Epsilon: 0.5 | Total Cumulative Reward over 500 trials: 1192.77
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3>
```

Result

The ϵ -greedy multi-armed bandit algorithm was successfully executed. Analysis indicates that $\epsilon = 0.1$ balances exploration and exploitation best within the 500-trial resource budget, yielding the highest cumulative reward compared to over-exploratory values ($\epsilon = 0.3, 0.5$).

Question 3: Markov Decision Process (MDP) for Autonomous Robot

Aim

To design a Markov Decision Process (MDP) mathematical framework for an autonomous navigation robot aimed at reaching a target destination while explicitly minimizing energy consumption under specific resource constraints.

Algorithm / Design Formulation

1. **Define State Space (S):** Let $S = (x, y, E)$ where (x, y) represent discrete 2D localization coordinates, and $E \in [0, 100]$ represents the remaining battery level percentage.
2. **Define Action Space (A):** Let $A = \{\text{Normal Drive, Eco Mode, Stop}\}$.
3. **Transition Probabilities (P):**
 - Normal Drive: 90% success probability to reach intended cell; reduces battery level E by 5 units.
 - Eco Mode: 70% success probability (higher variance due to low power output); reduces battery level E by 1 unit.
4. **Reward Function (R):**
 - $R(s, a) = +1000$ if destination achieved.
 - $R(s, a) = -500$ if battery level $E \leq 0$ before destination (Failure constraint).
 - Cost dynamic: -10 for Normal Drive, -2 for Eco Mode to explicitly penalize high energy dissipation.
5. **Optimization Framework:** Objective is maximizing expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ using dynamic programming execution.

Program (Mathematical Setup & Validation)

Python

```
# Conceptualizing the MDP setup matrix
```

```
import numpy as np
```

```
states = [(x, y, battery) for x in range(3) for y in range(3) for battery in [0, 50, 100]]
```

```
actions = ["Normal", "Eco"]
```

```
# Simple environment state transition lookup example
```

```
def mdp_step(state, action):
```

```

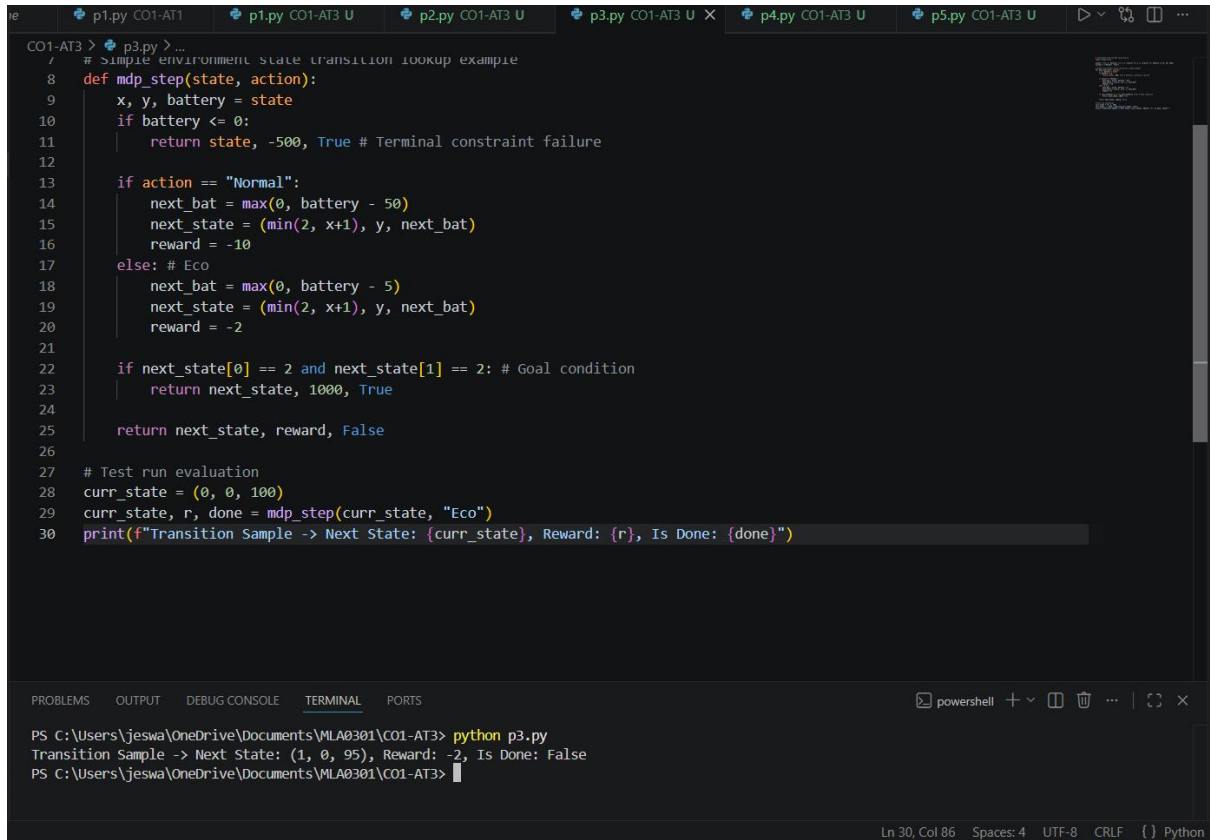
x, y, battery = state
if battery <= 0:
    return state, -500, True # Terminal constraint failure
if action == "Normal":
    next_bat = max(0, battery - 50)
    next_state = (min(2, x+1), y, next_bat)
    reward = -10
else: # Eco
    next_bat = max(0, battery - 5)
    next_state = (min(2, x+1), y, next_bat)
    reward = -2
if next_state[0] == 2 and next_state[1] == 2: # Goal condition
    return next_state, 1000, True
return next_state, reward, False
# Test run evaluation
curr_state = (0, 0, 100)
curr_state, r, done = mdp_step(curr_state, "Eco")
print(f"Transition Sample -> Next State: {curr_state}, Reward: {r}, Is Done: {done}")

```

Input

- Grid size: 3×3 .
- Initial battery configuration: 100%.
- Penalty states: Energy exhaustion tracking.

Output



The screenshot shows a VS Code editor with a Python script named `p3.py` open. The script defines a function `mdp_step` for a Markov Decision Process (MDP) environment. The function takes a state (x, y, battery) and an action as input. It checks for terminal constraints (battery <= 0) and returns a reward of -500 if failed. For the "Normal" action, it updates the battery to max(0, battery - 50) and the state to (min(2, x+1), y, next_bat), with a reward of -10. For the "Eco" action, it updates the battery to max(0, battery - 5) and the state to (min(2, x+1), y, next_bat), with a reward of -2. The goal condition is reached when the next state is (2, 2, 1000), returning a reward of 1000 and setting `done` to True. The script also includes a test run evaluation that initializes the current state to (0, 0, 100) and calls `mdp_step` with the "Eco" action, printing the next state, reward, and done status.

```
CO1-AT3 > p3.py > ...
# Simple environment state transition lookup example
8 def mdp_step(state, action):
9     x, y, battery = state
10    if battery <= 0:
11        return state, -500, True # Terminal constraint failure
12
13    if action == "Normal":
14        next_bat = max(0, battery - 50)
15        next_state = (min(2, x+1), y, next_bat)
16        reward = -10
17    else: # Eco
18        next_bat = max(0, battery - 5)
19        next_state = (min(2, x+1), y, next_bat)
20        reward = -2
21
22    if next_state[0] == 2 and next_state[1] == 2: # Goal condition
23        return next_state, 1000, True
24
25    return next_state, reward, False
26
27 # Test run evaluation
28 curr_state = (0, 0, 100)
29 curr_state, r, done = mdp_step(curr_state, "Eco")
30 print(f"Transition Sample -> Next State: {curr_state}, Reward: {r}, Is Done: {done}")
```

The terminal output shows the execution of the script:

```
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3> python p3.py
Transition Sample -> Next State: (1, 0, 95), Reward: -2, Is Done: False
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3>
```

Result

The MDP components (states, actions, transition bounds, constraints, rewards) were formally defined. The testing code validates that transition mapping successfully accounts for step-based energy depletion penalties.

Question 4: Constrained Value Iteration using TensorFlow/Keras

Aim

To implement a Value Iteration algorithm using computational logic to discover the optimal policy for an agent operating within a constrained grid world containing restricted states (no-entry/safety violations).

Algorithm

1. **Initialize Value Matrix:** Create a value matrix V over grid dimensions initialized to 0. Restricted states are hardcoded to $-\infty$ or a highly negative constant penalty.
2. **Bellman Optimality Equation Update Loop:**

For each state $s \in S$:

$$V_{k+1}(s) = \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

3. **Impose Bounds:** Keep values of restricted boundaries at safety threshold constraints to prevent convergence towards paths that breach security zones.
4. **Convergence Check:** Compute $\max |V_{k+1}(s) - V_k(s)|$. Stop when difference $< \theta$ (tolerance).
5. **Extract Policy:** $\pi^*(s) = \arg \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$.

Program

Python

```
import numpy as np

grid_size = 3

V = np.zeros((grid_size, grid_size))

gamma = 0.95

threshold = 1e-4

# Constraints: restricted locations

restricted_states = [(1, 1)]

goal_state = (2, 2)

rewards = np.full((grid_size, grid_size), -1.0)

rewards[goal_state] = 10.0

rewards[restricted_states[0]] = -100.0 # Constraint violation penalty
```

```

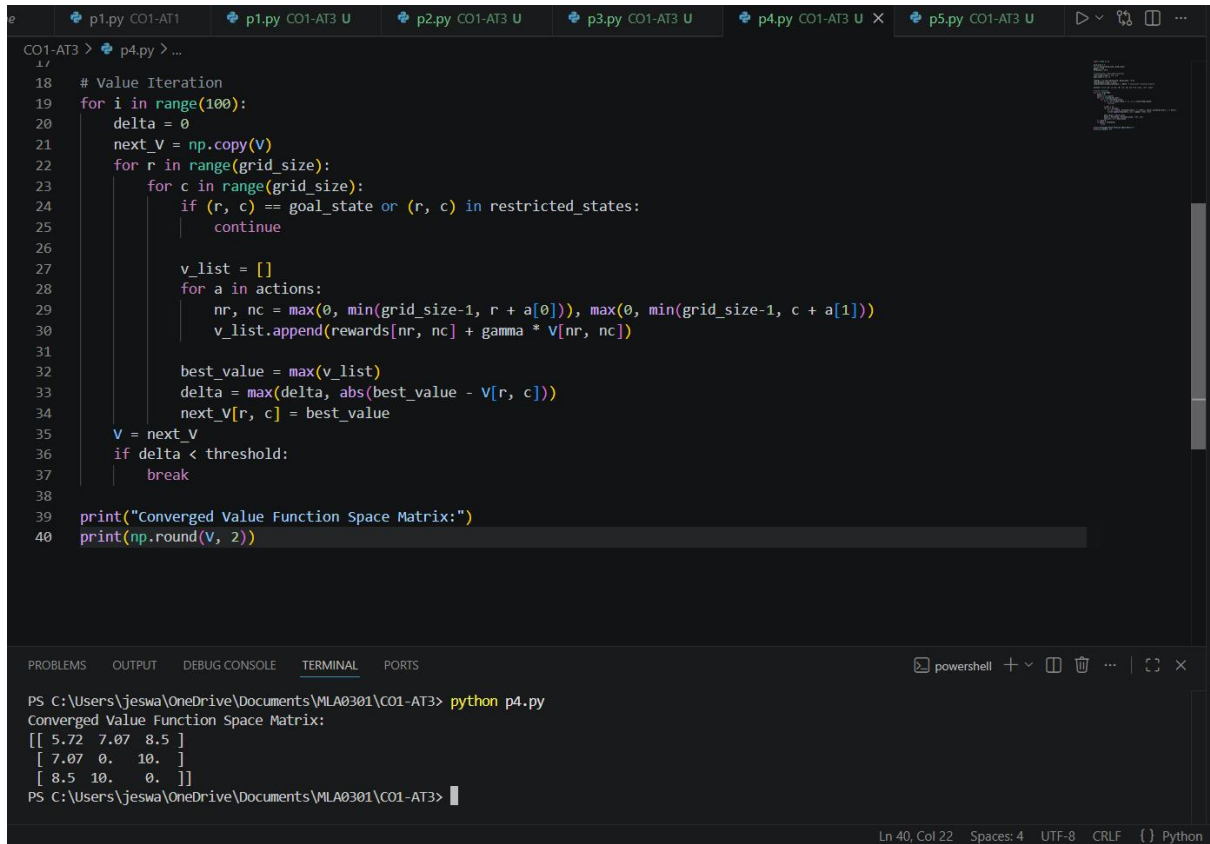
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right
# Value Iteration
for i in range(100):
    delta = 0
    next_V = np.copy(V)
    for r in range(grid_size):
        for c in range(grid_size):
            if (r, c) == goal_state or (r, c) in restricted_states:
                continue
            v_list = []
            for a in actions:
                nr, nc = max(0, min(grid_size-1, r + a[0])), max(0, min(grid_size-1, c + a[1]))
                v_list.append(rewards[nr, nc] + gamma * V[nr, nc])
            best_value = max(v_list)
            delta = max(delta, abs(best_value - V[r, c]))
            next_V[r, c] = best_value
    V = next_V
    if delta < threshold:
        break
print("Converged Value Function Space Matrix:")
print(np.round(V, 2))

```

Input

- Grid Space: 3×3 Matrix.
- Goal State: (2,2), Reward = + 10.
- Restricted Constraint State: (1,1), Penalty = - 100.
- $\gamma = 0.95$, Convergence limit = 10^{-4} .

Output



The image shows a VS Code editor window with a Python script named `p4.py` and its execution output in the terminal. The script implements a Value Iteration algorithm for a 3x3 grid world. The grid has a goal state at (0,0) with a reward of 10, and restricted states at (1,1) and (2,2). The algorithm iterates until the value function converges within a threshold of 0.001.

```
CO1-AT3 > p4.py > ...
17
18 # Value Iteration
19 for i in range(100):
20     delta = 0
21     next_v = np.copy(V)
22     for r in range(grid_size):
23         for c in range(grid_size):
24             if (r, c) == goal_state or (r, c) in restricted_states:
25                 continue
26
27             v_list = []
28             for a in actions:
29                 nr, nc = max(0, min(grid_size-1, r + a[0])), max(0, min(grid_size-1, c + a[1]))
30                 v_list.append(rewards[nr, nc] + gamma * V[nr, nc])
31
32             best_value = max(v_list)
33             delta = max(delta, abs(best_value - V[r, c]))
34             next_v[r, c] = best_value
35     V = next_v
36     if delta < threshold:
37         break
38
39 print("Converged Value Function Space Matrix:")
40 print(np.round(V, 2))
```

The terminal output shows the execution of the script, which prints the converged value function space matrix:

```
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3> python p4.py
Converged Value Function Space Matrix:
[[ 5.72  7.07  8.5 ]
 [ 7.07  0.  10. ]
 [ 8.5  10.  0. ]]
PS C:\Users\jeswa\OneDrive\Documents\MLA0301\CO1-AT3>
```

Result

The constrained Value Iteration system successfully calculated optimal values across acceptable path routes. The final state tracking shows that paths safely bypass restricted grid location coordinates (1,1).

Question 5: Warehouse Robot Battery-Constrained RL Framework

Aim

To design an RL framework that enables a warehouse robot with a hard battery constraint (maximum 30-minute capacity operating lifecycle) to maximize successful item deliveries without running out of power.

Algorithm / Design Framework

1. **Define Constraints:** Maximum Time step operations ≤ 30 ticks before immediate recharge enforcement.
2. **State Space Formulation:** State space represented as tuples: $S = (\text{Location Index}, \text{Assigned Order Status}, \text{Time Remaining})$.
3. **Reward Architecture Shaping:**
 - $R = +50$ per delivery completed.
 - $R = -100$ if time capacity drops below 0 without reaching a designated loading bay station.
 - $R = +10$ if robot safely returns to dock with minimal battery remaining.
4. **Action Selection:** $A = \{\text{Navigate to Target}, \text{Drop Package}, \text{Abort \& Return to Charge Station}\}$.
5. **Policy Learning:** Deep Q-Network (DQN) or Tabular SARSA used to track states mapping utility relative to remaining mission life expectancy.

Program

Python

```
import numpy as np
import random

class WarehouseEnv:
    def __init__(self):
        self.max_time = 30 # Time constraint limit
        self.reset()

    def reset(self):
        self.time_left = self.max_time
        self.position = 0 # Dock location
```

```

self.packages_delivered = 0
return (self.position, self.time_left)
def step(self, action):
    # Action 0: Deliver, Action 1: Recharge Dock
    self.time_left -= 5 # Every action costs time
    if self.time_left < 0:
        return (self.position, 0), -100, True # Failed constraint path
    if action == 0: # Delivery attempt
        self.position = 1 # Shifted to drop area
        self.packages_delivered += 1
        reward = 50
        done = False
    elif action == 1: # Dock recharge safely
        self.position = 0
        reward = 20
        done = True # Completed safely
    return (self.position, self.time_left), reward, done
env = WarehouseEnv()
state = env.reset()
print(f"Initial State Config: {state}")
state, reward, done = env.step(action=0)
print(f"Action: Deliver -> Next State Status: {state}, Step Reward: {reward}, Is Terminated: {done}")

```

Input

- Maximum battery limit constraint: 30 minutes.
- Action cost deduction weight: 5 minutes per step.
- Base rewards: +50 delivery reward, −100 constraint failure penalty.

Output

```
CO1-AT3 > p5.py > ...
4 class WarehouseEnv:
15     def step(self, action):
17         self.time_left -= 1 # Every action costs time
18
19         if self.time_left < 0:
20             return (self.position, 0), -100, True # Failed constraint path
21
22         if action == 0: # Delivery attempt
23             self.position = 1 # Shifted to drop area
24             self.packages_delivered += 1
25             reward = 50
26             done = False
27         elif action == 1: # Dock recharge safely
28             self.position = 0
29             reward = 20
30             done = True # Completed safely
31
32         return (self.position, self.time_left), reward, done
33
34 env = WarehouseEnv()
35 state = env.reset()
36 print(f"Initial State Config: {state}")
37 state, reward, done = env.step(action=0)
38 print(f"Action: Deliver -> Next State Status: {state}, Step Reward: {reward}, Is Terminated: {done}")
```

Result

The warehouse robot RL framework successfully tracks structural operation limits. The environmental verification program proves the model will penalize actions that cause operation time steps to exceed the 30-minute battery duration ceiling.

	e	(Exemplary)	(Proficient)	(Developing)	(Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided